

인공지능 개념 지식자료

AI Concepts: Relationships and Comparisons

개념 관계 · 비교 중심의 RAG 지식 문서 · 전 8장

개요

본 문서는 인공지능 입문 수준의 핵심 개념들을 관계와 차이 중심으로 설명하는 지식 자료이다. 개별 용어의 정의는 별도의 용어사전(glossary)이 담당하며, 이 문서는 개념 간 포함 관계, 혼동하기 쉬운 용어의 비교, 작동 원리, 선택 기준을 다룬다. 각 소제목 단위의 절은 다른 절에 의존하지 않고 독립적으로 읽을 수 있도록 작성되어, 검색 증강 생성(RAG)의 검색 단위(chunk)로 그대로 활용할 수 있다. 특정 강의, 교수, 학기에 종속되지 않는 표준 내용만을 담는다.

1장. 인공지능, 머신러닝, 딥러닝의 관계

세 개념의 포함 관계

인공지능(Artificial Intelligence, AI), 머신러닝(Machine Learning, ML), 딥러닝(Deep Learning, DL)은 동심원 형태의 포함 관계를 이룬다. 인공지능이 가장 넓은 개념으로, 사람의 지능적 작업을 기계로 구현하려는 모든 시도를 포괄한다. 머신러닝은 그중 데이터로부터 패턴을 학습하는 접근이며, 딥러닝은 머신러닝 중에서도 여러 층의 인공신경망을 사용하는 방법이다. 따라서 모든 딥러닝은 머신러닝이고 모든 머신러닝은 인공지능이지만, 그 역은 성립하지 않는다. 규칙 기반 전문가 시스템은 머신러닝이 아닌 인공지능의 예이고, 결정 트리는 딥러닝이 아닌 머신러닝의 예이다.

접근 방식의 차이: 규칙 작성과 패턴 학습

전통적 프로그래밍과 인공지능 초기의 규칙 기반 시스템에서는 사람이 판단 규칙을 직접 코드로 작성한다. 반면 머신러닝(Machine Learning)에서는 입력과 결과 데이터를 제공하면 모델이 규칙에 해당하는 패턴을 스스로 찾는다. 스팸 필터를 예로 들면, 규칙 기반은 '특정 단어가 있으면 스팸'이라는 조건을 사람이 나열하는 방식이고, 머신러닝은 스팸/정상이 표시된 대량의 메일에서 구분 기준을 모델이 학습하는 방식이다. 규칙이 복잡하거나 계속 변하는 문제일수록 학습 기반 접근이 유리하다.

딥러닝이 구분되는 지점: 특징 추출의 자동화

전통적 머신러닝에서는 모델에 입력할 특징(feature)을 사람이 설계했다. 예컨대 이미지 분류라면 색 분포나 윤곽 정보를 사람이 계산해 모델에 넣었다. 딥러닝(Deep Learning)은 이 특징 추출 단계까지 신경망이 스스로 수행한다는 점에서 구분된다. 원시 픽셀을 입력하면 낮은 층이 예지 같은 저수준 특징을, 높은 층이 물체 부분 같은 고수준 특징을 계층적으로 학습한다. 이 자동화 덕분에 이미지, 음성, 텍스트처럼 특징 설계가 어려운 데이터에서 딥러닝이 기존 방법을 크게 앞서게 되었다.

핵심 정리와 관련 용어

정리하면, 인공지능은 목표(지능의 구현), 머신러닝은 방법(데이터 기반 학습), 딥러닝은 그 방법의 한 갈래(심층 신경망)이다. 세 용어를 같은 뜻으로 혼용하면 안 되며, 특히 '인공지능 = 딥러닝'이라는 등식은 성립하지 않는다. 관련 용어: 인공지능(AI), 머신러닝(ML), 딥러닝(DL), 인공신경망(ANN), 전문가 시스템.

2장. 지도학습, 비지도학습, 강화학습의 차이

세 학습 방식을 가르는 기준: 피드백의 형태

머신러닝의 세 학습 방식은 모델이 받는 피드백의 형태로 구분된다. 지도학습(Supervised Learning)은 각 입력마다 정답 레이블을 즉시 제공받는다. 비지도학습(Unsupervised Learning)은 아무런 정답 없이 데이터 자체의 구조를 찾는다. 강화학습(Reinforcement Learning, RL)은 정답 대신 행동의 결과로 주어지는 보상(reward) 신호를 받으며, 그 보상은 종종 여러 행동이 끝난 뒤에야 주어진다. 어떤 피드백을 확보할 수 있는지가 문제에 맞는 학습 방식을 고르는 첫 번째 기준이 된다.

지도학습의 두 과제: 분류와 회귀

지도학습의 과제는 출력의 형태에 따라 둘로 나뉜다. 분류(Classification)는 스팸/정상, 개/고양이처럼 정해진 범주 중 하나를 고르는 문제이고, 회귀(Regression)는 집값이나 기온처럼 연속적인 수치를 예측하는 문제이다. 같은 데이터라도 질문에 따라 과제가 달라진다. 예를 들어 학생 성적 데이터에서 '합격 여부'를 예측하면 분류, '점수'를 예측하면 회귀이다. 손실 함수도 이에 따라 달라져, 분류에는 교차 엔트로피가, 회귀에는 평균제곱오차(MSE)가 주로 쓰인다.

비지도학습의 두 과제: 군집화와 차원 축소

비지도학습의 대표 과제는 군집화(Clustering)와 차원 축소(Dimensionality Reduction)이다. 군집화는 비슷한 데이터끼리 그룹으로 묶는 것으로, 구매 이력 기반 고객 세분화가 전형적인 예이다. 차원 축소는 데이터의 중요한 정보를 유지하면서 변수의 수를 줄이는 것으로, 시각화나 저장 효율화에 쓰인다. 분류와 군집화는 '그룹을 나눈다'는 점이 비슷해 혼동하기 쉬운데, 분류는 미리 정의된 그룹에 배정하는 지도학습이고, 군집화는 그룹 자체를 데이터에서 발견하는 비지도학습이라는 점이 다르다.

강화학습의 구조: 에이전트, 환경, 보상

강화학습에서는 에이전트(agent)가 환경(environment)의 상태를 관찰해 행동을 선택하고, 환경은 다음 상태와 보상을 돌려준다. 에이전트의 목표는 한 번의 보상이 아니라 장기적인 누적 보상을 최대화하는 행동 규칙, 즉 정책(policy)을 찾는 것이다. 바둑에서 승리라는 보상은 수백 수가 끝난 뒤에야 주어지므로, 어떤 수가 좋았는지를 거슬러 판단해야 한다. 이런 지연된 보상과, 새로운 시도(탐험)와 검증된 행동(활용) 사이의 균형이 강화학습 특유의 난제이다. 지도학습과 달리 각 행동의 정답을 아무도 알려주지 않는다는 점이 근본적 차이이다.

핵심 정리와 관련 용어

요약하면, 정답이 있으면 지도학습, 정답 없이 구조를 찾으면 비지도학습, 보상으로 행동을 배우면 강화학습이다. 실무에서는 레이블 확보 비용이 방식 선택을 좌우하는 경우가 많다. 관련 용어: 지도학습, 비지도학습, 강화학습(RL), 분류, 회귀, 군집화, 차원 축소, 보상, 정책.

3장. 모델 학습 과정: 데이터, 손실 함수, 경사하강법, 학습률

학습의 정의: 손실을 줄이는 파라미터 찾기

머신러닝에서 학습이란 손실 함수(Loss Function)의 값을 최소화하는 모델 파라미터를 찾는 과정이다. 손실 함수는 모델의 예측과 정답의 차이를 하나의 수치로 나타내며, 이 수치가 작을수록 좋은 모델이다. 즉 '좋은 모델'이라는 막연한 목표가 '손실 최소화'라는 수학적 목표로 번역되고, 그 덕분에 최적화 알고리즘을 적용할 수 있게 된다. 어떤 손실 함수를 고르느냐가 모델이 무엇을 잘하도록 학습될지를 결정한다.

데이터 분할: 훈련, 검증, 테스트

모델을 공정하게 평가하려면 데이터를 세 묶으로 나눈다. 훈련 데이터(training set)로 파라미터를 학습하고, 검증 데이터(validation set)로 학습률 같은 하이퍼파라미터를 조정하며, 테스트 데이터(test set)로 최종 성능을 한 번만 측정한다. 테스트 데이터를 하이퍼파라미터 조정에 반복 사용하면 그 데이터에 우연히 맞는 설정을 고르게 되어, 최종 평가가 실제 성능보다 낙관적으로 나오는 문제가 생긴다. 검증과 테스트를 분리하는 이유가 여기에 있다.

경사하강법의 원리: 기울기 반대 방향으로 한 걸음씩

경사하강법(Gradient Descent, GD)은 손실 함수를 최소화하는 대표적 최적화 방법이다. 현재 파라미터 위치에서 손실의 기울기를 계산하고, 손실이 감소하는 기울기 반대 방향으로 파라미터를 이동시키는 일을 반복한다. 갱신식은 '새 파라미터 = 현재 파라미터 - 학습률 X 기울기'로 쓸 수 있다. 안개 낀 산에서 발밑 경사만 보고 내리막으로 한 걸음씩 내려가는 것에 비유된다. 이 비유는 한계도 보여주는데, 눈앞의 내리막만 따라가므로 전체에서 가장 낮은 지점(전역 최솟값)이 아니라 주변에서만 낮은 지점(지역 최솟값)에 도착할 수 있다.

학습률이 너무 클 때와 작을 때

학습률(Learning Rate, LR)은 경사하강법에서 한 걸음의 보폭이다. 학습률이 너무 크면 최솟값 근처를 계속 건너뛰어 손실이 요동치거나 오히려 커지는 발산이 일어난다. 반대로 너무 작으면 수렴까지 지나치게 오래 걸리고 얇은 지역 최솟값에 일찍 갇힐 수 있다. 그래서 학습률은 가장 먼저, 가장 신중하게 조정하는 하이퍼파라미터이다. 실무에서는 학습 초반에 크게 시작해 점차 줄이는 스케줄링이나, 기울기 이력에 따라 보폭을 자동 조절하는 Adam 같은 적응형 최적화 기법을 함께 사용한다.

역전파와 경사하강법의 역할 분담

신경망 학습에서 역전파(Backpropagation)와 경사하강법은 혼동하기 쉽지만 역할이 다르다. 역전파는 연쇄 법칙을 이용해 각 가중치에 대한 손실의 기울기를 효율적으로 '계산'하는 알고리즘이고, 경사하강법은 그 기울기를 받아 가중치를 '갱신'하는 최적화 방법이다. 즉 한 번의 학습 단계는 순전파(예측과 손실 계산), 역전파(기울기 계산), 경사하강법(가중치 갱신)의 순서로

이루어진다. 관련 용어: 손실 함수, 경사하강법(GD), 학습률(LR), 하이퍼파라미터, 역전파, 훈련/검증/테스트 데이터.

4장. 과적합과 과소적합, 그리고 정규화 기법

과적합과 과소적합: 증상으로 구분하기

과적합(Overfitting)은 모델이 훈련 데이터의 잡음까지 외워버려, 훈련 성능은 높지만 새 데이터에서 성능이 떨어지는 현상이다. 과소적합(Underfitting)은 모델이 너무 단순해 훈련 데이터의 패턴조차 잡지 못해, 훈련 성능부터 낮은 현상이다. 구분법은 명확하다. 훈련 성능이 낮으면 과소적합이고, 훈련 성능은 높는데 검증 성능만 낮으면 과적합이다. 학생에 비유하면 과적합은 기출 답만 통째로 외운 학생, 과소적합은 아직 공부량 자체가 부족한 학생이다.

학습 곡선으로 과적합 진단하기

학습이 진행되는 동안 훈련 손실과 검증 손실을 함께 그린 학습 곡선은 과적합 진단의 기본 도구이다. 건강한 학습에서는 두 손실이 함께 내려간다. 과적합이 시작되면 훈련 손실은 계속 내려가는데 검증 손실이 바닥을 찍고 다시 올라가는 갈림이 나타난다. 이 갈림 지점이 학습을 멈추기에 적절한 시점이며, 검증 손실이 개선되지 않으면 학습을 중단하는 조기 종료(Early Stopping)는 이를 자동화한 기법이다.

정규화(Regularization)와 Normalization의 구분

정규화라는 한국어 번역은 서로 다른 두 개념에 함께 쓰여 혼동을 일으킨다.

Regularization(규제)은 과적합을 줄이기 위해 모델 복잡도에 제약을 가하는 기법의 총칭이다.

반면 Normalization(정규화)은 입력값이나 층 출력의 스케일과 분포를 조정하는 것으로, min - max 정규화나 배치 정규화(Batch Normalization)가 여기에 속한다. 전자는 '모델이 외우지 못하게 막는 것', 후자는 '값의 범위를 다듬는 것'으로 목적이 다르다. 문서나 강의에서 '정규화'가 나오면 어느 쪽을 뜻하는지 문맥으로 확인해야 한다.

대표적인 과적합 대응 기법들

과적합에 대한 대응은 크게 네 갈래이다. 첫째, 드롭아웃(Dropout)은 학습 중 뉴런을 무작위로 꺼서 특정 뉴런 조합에 대한 의존을 깨뜨린다. 둘째, 가중치 감쇠(Weight Decay)는 손실 함수에 가중치 크기에 대한 벌점을 더해 모델이 극단적인 가중치를 갖지 못하게 한다. 셋째, 조기 종료는 검증 성능이 나빠지기 전에 학습을 멈춘다. 넷째, 데이터 증강(Data Augmentation)은 이미지 회전이나 자르기처럼 데이터를 변형해 훈련 데이터의 다양성을 늘린다. 모두 훈련 성능을 조금 양보하는 대신 일반화 성능을 얻는다는 공통점이 있다.

핵심 정리와 관련 용어

과적합과 과소적합은 훈련 성능과 검증 성능의 조합으로 진단하고, 과적합에는 정규화(규제) 기법으로, 과소적합에는 모델 확장이나 학습 연장으로 대응한다. 관련 용어: 과적합, 과소적합, 정규화(Regularization), 드롭아웃, 가중치 감쇠, 조기 종료, 데이터 증강, 배치 정규화(Normalization), 학습 곡선.

5장. 인공신경망: 퍼셉트론, 활성화 함수, 역전파

퍼셉트론: 신경망의 최소 단위

퍼셉트론(Perceptron)은 인공신경망(Artificial Neural Network, ANN)의 기본 단위이다. 여러 입력에 각각 가중치(weight)를 곱해 모두 더하고 편향(bias)을 더한 값을 활성화 함수에 통과시켜 출력을 만든다. 수식으로는 '출력 = 활성화함수(가중치와 입력의 곱의 합 + 편향)'이다. 여기서의 편향은 가중합을 이동시키는 학습 파라미터로, AI 윤리에서 말하는 데이터 편향과는 전혀 다른 개념이므로 주의해야 한다. 단층 퍼셉트론은 직선으로 나눌 수 있는 문제만 풀 수 있어 XOR 문제를 해결하지 못했고, 이 한계는 은닉층을 추가한 다층 퍼셉트론(MLP)으로 극복되었다.

활성화 함수: 왜 비선형이 필요한가

활성화 함수(Activation Function)가 없다면 층을 아무리 쌓아도 전체 신경망은 하나의 선형 변환과 같아진다. 선형 변환의 합성은 여전히 선형이기 때문이다. 비선형 활성화 함수가 층 사이에 끼어들어야 층을 쌓는 것이 표현력 증가로 이어진다. 초기에는 출력을 0~1로 누르는 시그모이드(Sigmoid)와 -1~1로 누르는 tanh가 쓰였으나, 이들은 입력이 크거나 작을 때 기울기가 거의 0이 되어 깊은 망에서 학습을 방해했다. 현재는 음수를 0으로 만들고 양수를 그대로 통과시키는 ReLU와 그 변형들이 은닉층의 표준이다.

역전파와 기울기 소실

역전파(Backpropagation)는 출력층의 오차로부터 시작해 연쇄 법칙으로 각 층 가중치의 기울기를 입력층 방향으로 차례로 계산한다. 이때 각 층의 기울기가 곱해지며 뒤로 전달되는데, 시그모이드처럼 기울기 최댓값이 1보다 작은 함수를 쓰면 1보다 작은 수가 층 수만큼 곱해져 앞쪽 층의 기울기가 사실상 0이 된다. 이것이 기울기 소실(Vanishing Gradient)이며, 깊은 신경망 학습을 오래 가로막았던 문제이다. ReLU 계열 활성화 함수, 배치 정규화, 그리고 층을 건너뛰는 지름길을 두는 잔차 연결(Residual Connection)이 이 문제를 완화해 수백 층 규모의 학습을 가능하게 했다.

핵심 정리와 관련 용어

신경망은 퍼셉트론이라는 단위를 층으로 쌓은 구조이고, 활성화 함수가 그 쌓임에 의미를 부여하며, 역전파가 학습을 가능하게 한다. 깊이가 주는 표현력과 기울기 소실이라는 대가 사이의 싸움이 딥러닝 구조 발전의 한 축이었다. 관련 용어: 퍼셉트론, 다층 퍼셉트론(MLP), 활성화 함수, ReLU, 시그모이드, 역전파, 기울기 소실, 잔차 연결.

6장. CNN의 핵심 구조와 컴퓨터 비전 과제

왜 이미지에는 CNN인가

이미지를 완전 연결 신경망으로 처리하면 픽셀 하나하나가 모든 뉴런과 연결되어 파라미터가 폭발하고, 물체가 몇 픽셀만 이동해도 입력이 전혀 다른 것으로 취급된다. 합성곱 신경망(Convolutional Neural Network, CNN)은 작은 필터를 이미지 전체에서 재사용하는 파라미터 공유로 이 두 문제를 해결한다. 같은 필터가 모든 위치를 훑으므로 파라미터 수가 크게 줄고, 물체가 어디에 있든 같은 기준으로 탐지된다. 이미지의 공간 구조라는 사전 지식을 구조에 새긴 설계라 할 수 있다.

합성곱과 풀링의 역할 분담

합성곱(Convolution)과 풀링(Pooling)은 역할이 다르다. 합성곱은 학습된 필터로 특징을 '찾는' 연산으로, 필터가 입력 위를 스트라이드(stride) 간격으로 이동하며 각 위치의 반응 강도를 특징 맵(Feature Map)에 기록한다. 풀링은 찾은 특징을 '요약하는' 연산으로, 영역별 최대값(최대 풀링) 등으로 특징 맵을 축소해 계산량을 줄이고 작은 위치 변화에 둔감하게 만든다. 합성곱 필터의 가중치는 학습되지만 풀링에는 학습되는 가중치가 없다는 점도 중요한 차이이다. 현대 구조에서는 풀링 대신 스트라이드가 큰 합성곱으로 축소를 대신하기도 한다.

특징의 계층: 에지에서 물체로

CNN의 층을 따라 특징 맵이 담는 내용은 점점 추상화된다. 입력에 가까운 층의 필터는 에지, 색 대비, 단순한 질감에 반응하고, 중간 층은 이를 조합한 무늬나 부품(눈, 바퀴 등)에, 깊은 층은 물체 전체나 장면 수준의 개념에 반응한다. 사람이 특징의 위계를 설계하지 않아도 데이터로부터 이런 계층이 형성된다는 것이 딥러닝 표현 학습의 핵심이며, 낮은 층의 범용적 특징은 다른 과제에 재사용(전이 학습)되기도 한다.

컴퓨터 비전의 세 과제: 분류, 탐지, 분할

컴퓨터 비전의 대표 과제는 요구하는 답의 정밀도로 구분된다. 이미지 분류(Image Classification)는 '무엇이 있는가'만 답한다. 객체 탐지(Object Detection)는 '무엇이 어디에 몇 개 있는가'까지 답하며, 물체마다 클래스와 경계 상자를 출력한다. 이미지 분할(Segmentation)은 픽셀 단위로 어느 픽셀이 어느 물체에 속하는지까지 구분하는 가장 세밀한 과제이다. 자율주행 예로 들면, 분류는 '보행자가 있다', 탐지는 '보행자가 저 위치에 있다', 분할은 '이 픽셀들이 보행자다'에 해당한다. 관련 용어: CNN, 합성곱, 풀링, 특징 맵, 스트라이드, 패딩, 객체 탐지, 분할.

7장. 자연어 처리의 발전: RNN에서 어텐션, 트랜스포머로

텍스트가 벡터가 되기까지: 토큰화와 임베딩

신경망은 문자를 직접 다루지 못하므로, 자연어 처리(NLP)의 첫 단계는 텍스트의 수치화이다. 먼저 토큰화(Tokenization)가 문장을 단어나 부분 단어(subword) 단위로 자르고 각 토큰에 번호를 붙인다. 다음으로 임베딩(Embedding)이 각 토큰을 수백 차원의 실수 벡터로 바꾼다. 학습 목적에 따라 의미나 쓰임이 비슷한 토큰이 가까운 벡터로 표현될 수 있어, 벡터 연산으로 의미를 다룰 수 있게 된다. 토큰화는 '자르는' 과정이고 임베딩은 '의미 벡터로 바꾸는' 과정이라는 점에서 서로 다른 단계이다. 한국어는 조사와 어미가 붙는 교착어라 토큰화 방식이 성능에 특히 크게 작용한다.

RNN의 아이디어와 장기 의존성 문제

순환 신경망(Recurrent Neural Network, RNN)은 문장을 앞에서부터 한 토큰씩 읽으며, 지금까지 읽은 내용의 요약인 은닉 상태를 갱신해 나간다. 순서라는 언어의 본질을 구조에 반영한 자연스러운 설계였지만, 시퀀스가 길어지면 앞부분의 정보가 뒤까지 전달되지 못하는 장기 의존성(long-term dependency) 문제가 드러났다. 문장 첫머리의 주어를 문장 끝에서 기억하지 못하는 식이다. 게이트로 정보 흐름을 조절하는 LSTM과 GRU가 이를 상당히 완화했지만, 앞 토큰을 처리해야 다음 토큰을 처리할 수 있는 순차 구조 탓에 병렬화가 어렵다는 한계는 그대로 남았다.

어텐션: 필요한 곳을 직접 참조한다

어텐션(Attention)은 정보를 순서대로 전달하는 대신, 필요한 위치를 직접 참조하는 발상의 전환이다. 처리 중인 요소의 질의(Query) 벡터와 각 위치의 키(Key) 벡터의 유사도로 관련도를 계산하고, 그 가중치로 값(Value) 벡터들을 가중 평균해 출력을 만든다. 멀리 떨어진 단어라도 관련도만 높으면 한 번에 연결되므로 장기 의존성 문제가 구조적으로 해소된다. 하나의 시퀀스가 자기 자신을 참조하면 셀프 어텐션(Self-Attention), 번역처럼 다른 시퀀스를 참조하면 크로스 어텐션이라 부른다.

트랜스포머: 어텐션만으로 충분하다

트랜스포머(Transformer)는 순환 구조를 완전히 제거하고 셀프 어텐션만으로 시퀀스를 처리하는 구조로, 2017년 'Attention Is All You Need' 논문에서 제안되었다. 모든 토큰 쌍의 관계를 한 번에 계산하므로 시퀀스 전체를 병렬로 처리할 수 있고, 이것이 GPU 기반 대규모 학습과 맞물려 모델 규모의 급격한 확대를 가능하게 했다. 순서 정보는 위치 인코딩으로 별도 주입하며, 서로 다른 관점의 관계를 병렬로 포착하는 멀티헤드 어텐션을 사용한다. RNN과의 차이를 한 줄로 요약하면, RNN은 '순서대로 기억을 이어가는' 구조이고 트랜스포머는 '전체를 펼쳐 놓고 관계를 계산하는' 구조이다. 관련 용어: 토큰화, 임베딩, RNN, LSTM, 어텐션, 셀프 어텐션, 트랜스포머, 위치 인코딩.

8장. 대형 언어 모델: 사전 학습, 미세 조정, 프롬프트, RAG

LLM의 기본기: 다음 토큰 예측과 규모의 법칙

대형 언어 모델(Large Language Model, LLM)은 방대한 텍스트에서 앞 토큰들로 다음 토큰을 예측하는 과제를 대규모로 반복 학습한 모델이며, 현대의 대부분은 트랜스포머 구조를 기반으로 한다. 이 단순한 과제를 잘하려면 문법, 지식, 추론 패턴까지 익혀야 하므로, 다음 토큰 예측은 언어 능력 전반을 학습시키는 수단이 된다. 모델과 데이터의 규모가 커질수록 성능이 꾸준히 향상되는 규모의 법칙(Scaling Law)이 관찰되었고, 일정 규모를 넘으면 따로 가르치지 않은 능력이 나타나기도 한다.

모델을 작업에 맞추는 두 경로: 미세 조정과 프롬프트

사전 학습(Pre-training)된 범용 모델을 특정 용도에 맞추는 경로는 둘이다. 미세 조정(Fine-tuning)은 목적 데이터로 파라미터를 추가 학습해 지식과 스타일을 모델 내부에 새긴다. 프롬프트(Prompt) 기반 접근은 파라미터를 건드리지 않고 지시문과 예시로 원하는 동작을 유도하며, 예시만으로 작업을 수행하는 모델의 능력을 인컨텍스트 학습(In-Context Learning, ICL)이라 한다. 미세 조정은 일관된 형식·스타일이 필요할 때 강력하지만 갱신마다 재학습이 필요하고, 프롬프트는 비용이 거의 없지만 문맥 길이의 제약을 받는다. 실무에서는 두 경로를 조합해 쓴다.

환각: 유창함은 사실성을 보장하지 않는다

환각(Hallucination)은 LLM이 사실이 아닌 내용을 그럴듯한 문체로 생성하는 현상이다. 모델은 '이어질 확률이 높은 토큰'을 만들 뿐 문장의 사실 여부를 검증하지 않으므로, 지식이 없는 질문에도 유창한 답을 지어낼 수 있다. 존재하지 않는 논문을 인용하거나 수치를 틀리게 말하는 것이 전형적이며, 학습 데이터가 적은 전문 영역과 학습 시점 이후의 최신 정보에서 특히 잦다. 유창함과 사실성이 별개라는 이 성질 때문에, 신뢰가 필요한 응용에는 근거를 함께 제공하는 장치가 요구된다.

RAG: 검색으로 근거를 보강하는 생성

검색 증강 생성(Retrieval-Augmented Generation, RAG)은 환각과 최신 정보 부재를 함께 겨냥한 기법이다. 사전에 문서를 청크로 나눠 임베딩으로 변환해 벡터 데이터베이스(Vector Database)에 저장해 두고, 질의가 오면 유사한 청크를 검색해 프롬프트에 붙여 모델에 전달한다. 모델은 검색된 근거를 바탕으로 답을 생성하므로 출처 제시가 가능해지고, 지식베이스만 갱신하면 재학습 없이 최신 정보가 반영된다. 다만 검색이 잘못되면 틀린 근거가 답을 오염시키므로, 임베딩 모델의 품질과 청킹 전략이 시스템 전체의 품질을 좌우한다.

미세 조정과 RAG는 언제 각각 선택하는가

미세 조정과 RAG는 혼동하기 쉽지만 지식이 놓이는 위치가 다르다. 미세 조정은 지식을 모델 파라미터 '안에' 새기고, RAG는 지식을 모델 '밖의' 지식베이스에 두고 필요할 때 검색한다. 자주 바뀌는 정보(강의자료, 사내 문서, 최신 뉴스)에는 갱신이 쉬운 RAG가 적합하고, 말투 · 형식 · 도메인 감각처럼 일관되게 몸에 배야 하는 능력에는 미세 조정이 적합하다. 근거 추적이 필요하면 RAG가 유리하며, 두 방법을 함께 쓰는 경우도 많다. 관련 용어: LLM, 사전 학습, 미세 조정, 프롬프트, 인컨텍스트 학습(ICL), 환각, RAG, 벡터 데이터베이스, 임베딩.

9장. 고전 머신러닝 알고리즘

회귀와 분류를 가르는 기준: 출력의 형태

고전 머신러닝 알고리즘은 먼저 출력의 형태로 나뉜다. 선형 회귀(Linear Regression)는 입력의 가중 합으로 집값이나 기온 같은 연속적인 수치를 예측하는 회귀 모델이다. 로지스틱 회귀(Logistic Regression)는 이름에 '회귀'가 있지만 실제로는 분류 모델로, 가중 합에 시그모이드를 적용해 확률을 내고 임계값으로 클래스를 정한다. 이 이름의 함정이 입문자가 가장 자주 혼동하는 지점이다. 회귀는 '얼마인가'를, 분류는 '어느 범주인가'를 예측한다고 기억하면 된다.

결정 트리와 앙상블: 하나에서 여럿으로

결정 트리(Decision Tree)는 특징에 대한 질문으로 데이터를 반복해 나누며 예측하는 모델로, 규칙처럼 해석할 수 있어 설명력이 높지만 깊어지면 과적합하기 쉽다. 이 약점을 여러 트리를 묶어 보완한 것이 앙상블(Ensemble)이다. 랜덤 포레스트(Random Forest)는 데이터와 특징을 무작위로 뽑아 만든 여러 트리를 독립적으로 학습해 다수결로 종합하는 배경 방식이고, 부스팅(Boosting)은 이전 트리가 틀린 부분을 다음 트리가 집중해 보완하도록 순차적으로 학습한다. 랜덤 포레스트는 병렬적이고 안정적이며, 부스팅은 순차적이고 성능이 더 높지만 잡음에 민감하다는 차이가 있다.

거리 기반과 확률 기반: KNN, SVM, 나이브 베이즈

몇몇 고전 알고리즘은 서로 다른 원리로 분류한다. k-최근접 이웃(KNN)은 새 데이터와 가까운 k개 이웃의 다수결로 예측하며, 학습 과정 없이 예측 때 계산하는 게으른 학습이다. 서포트 벡터 머신(SVM)은 두 클래스 사이의 여백(마진)을 최대로 하는 경계를 찾고, 커널로 비선형 분류도 처리한다. 나이브 베이즈(Naive Bayes)는 베이즈 정리에 특징 독립 가정을 더해 확률로 분류하며 매우 빠르다. 이름과 개념이 비슷한 KNN과 k-평균을 혼동하기 쉬운데, KNN은 레이블을 예측하는 지도학습이고 k-평균은 레이블 없이 군집을 찾는 비지도학습이다.

비지도 알고리즘: 군집화와 차원 축소

정답 레이블이 없는 데이터에는 비지도 알고리즘을 쓴다. k-평균 군집화(k-Means)는 데이터를 k개의 군집으로 나누되 각 군집 중심과의 거리 합을 최소화하며, 군집 수 k를 사람이 미리 정해야 한다. 주성분 분석(PCA)은 데이터의 분산을 가장 잘 설명하는 축을 찾아 정보 손실을 최소화하며 차원을 줄이는 대표적 차원 축소 기법이다. 군집화가 '데이터를 그룹으로 묶는' 것이라면, 차원 축소는 '변수의 수를 줄이는' 것으로 목적이 다르다. 관련 용어: 선형 회귀, 로지스틱 회귀, 결정 트리, 랜덤 포레스트, 부스팅, SVM, KNN, 나이브 베이즈, k-평균, PCA.

10장. 모델 평가와 검증

정확도의 함정과 정밀도 · 재현율

정확도(Accuracy)는 전체 예측 중 맞힌 비율로 가장 직관적이지만, 클래스가 치우친 데이터에서는 오해를 부른다. 99%가 정상인 데이터에서 모두 정상이라 찍어도 정확도 99%가 나오기 때문이다. 이때는 정밀도와 재현율(Precision and Recall)을 봐야 한다. 정밀도는 양성이라 예측한 것 중 실제 양성 비율로 '거짓 양성을 줄였는가'를, 재현율은 실제 양성 중 찾아낸 비율로 '놓친 양성이 적은가'를 본다. 둘은 대개 상충해서, 암 진단처럼 놓치면 안 되는 문제는 재현율을, 스팸 분류처럼 정상 메일을 지켜야 하는 문제는 정밀도를 중시한다.

F1 점수와 혼동 행렬

정밀도와 재현율을 하나의 수치로 요약할 때 F1 점수(F1 Score)를 쓴다. F1은 두 값의 조화평균이라 어느 하나라도 낮으면 크게 떨어지므로, 정밀도와 재현율이 모두 높아야 F1이 높다. 이 모든 지표의 계산 근거가 되는 것이 혼동 행렬(Confusion Matrix)이다. 혼동 행렬은 실제와 예측을 교차해 참양성 · 거짓양성 · 거짓음성 · 참음성의 개수를 정리한 표로, 정확도 · 정밀도 · 재현율 · F1이 전부 이 네 값에서 나온다. 또한 어떤 클래스를 어떤 클래스로 자주 헛갈리는지 오류의 패턴까지 보여준다.

교차 검증: 한 번의 분할을 믿지 않는다

데이터를 훈련 · 검증 · 테스트로 한 번 나누면, 그 분할이 우연히 유리하거나 불리하게 나뉘어 성능 추정이 흔들릴 수 있다. 교차 검증(Cross - Validation)은 데이터를 k개 조각으로 나눈 뒤 검증 조각을 바꿔가며 k번 학습 · 평가하고 결과를 평균해 이 문제를 줄인다. 성능 추정이 안정적이고 데이터를 효율적으로 쓸 수 있어, 특히 데이터가 적을 때와 하이퍼파라미터를 고를 때 유용하다. 다만 여러 번 학습하므로 계산 비용이 늘어난다. 관련 용어: 정확도, 정밀도, 재현율, F1 점수, 혼동 행렬, 교차 검증, 과적합.

11장. 최적화 심화: 경사하강법의 여러 형태

배치 · 확률적 · 미니배치 경사하강법

경사하강법은 기울기를 몇 개의 데이터로 계산하느냐에 따라 갈린다. 배치 경사하강법은 매 갱신마다 전체 데이터를 써서 정확하지만 느리다. 확률적 경사하강법(SGD)은 한 번에 하나나 소수의 샘플만으로 기울기를 추정해 훨씬 자주, 빠르게 갱신하지만 경로에 잡음이 있다. 그 절충이 미니배치(Mini - batch)로, 수십에서 수백 개를 묶어 속도와 안정성의 균형을 잡으며 GPU 병렬 연산과도 잘 맞아 딥러닝의 표준이 되었다. 여기서 미니배치는 한 번 갱신에 쓰는 묶음이고, 에폭(Epoch)은 전체 데이터를 한 번 모두 사용한 단위라는 점을 구분해야 한다.

학습을 제어하는 하이퍼파라미터: 학습률과 배치 크기

학습 과정은 여러 하이퍼파라미터로 조절된다. 학습률(Learning Rate)은 한 걸음의 보폭으로, 너무 크면 발산하고 너무 작으면 수렴이 느리다. 배치 크기(Batch Size)는 한 번 갱신에 쓰는 샘플 수로, 크면 기울기 추정이 안정적이고 GPU 효율이 좋지만 메모리를 많이 쓰고, 작으면 갱신이 잦고 잡음이 많다. 학습률과 배치 크기는 서로 영향을 주므로 함께 조정해야 한다. 에폭 수까지 더해 이 셋이 학습을 제어하는 대표적 하이퍼파라미터이다.

모멘텀과 적응형 최적화

단순 경사하강법은 좁고 긴 골짜기에서 지그재그로 진동하며 느리게 수렴한다.

모멘텀(Momentum)은 공이 관성을 갖고 굴러가듯 과거 기울기 방향을 누적해, 같은 방향은 가속하고 진동은 상쇄해 수렴을 빠르고 안정적으로 만든다. 여기에 학습률을 기울기 이력에 따라 자동 조절하는 아이디어를 결합한 것이 Adam 같은 적응형 최적화 기법으로, 오늘날 딥러닝 학습에 널리 쓰인다. 관련 용어: 경사하강법, 확률적 경사하강법(SGD), 미니배치, 에폭, 학습률, 배치 크기, 모멘텀.

12장. 탐색 알고리즘: 무정보 탐색과 정보 탐색

무정보 탐색: 너비 우선과 깊이 우선

무정보 탐색은 목표까지의 거리 정보 없이 상태 공간을 훑는다. 너비 우선 탐색(BFS)은 시작점에서 가까운 층부터 넓게 탐색하며 큐를 쓰고, 모든 간선 비용이 같다면 최단 경로를 보장하지만 저장할 노드가 많아 메모리 부담이 크다. 깊이 우선 탐색(DFS)은 한 갈래를 끝까지 파고들다 막히면 되돌아오며 스택이나 재귀를 쓰고, 메모리는 적게 쓰지만 최단 경로를 보장하지 않고 무한 깊이나 순환에 빠질 수 있다. 넓게 보되 메모리를 쓸 것인가, 깊게 보되 최적성을 포기할 것인가의 선택이다.

비용을 고려한 무정보 탐색: 균일 비용 탐색

간선마다 비용이 다르다면 단순히 층수로만 판단할 수 없다. 균일 비용 탐색(Uniform - Cost Search)은 시작점에서의 누적 경로 비용이 가장 작은 노드를 우선순위 큐로 먼저 확장해, 비용이 제각각일 때도 최소 비용 경로를 보장한다. 모든 간선 비용이 같으면 너비 우선 탐색과 같아진다. 다만 목표까지의 거리를 추정하는 정보를 쓰지 않아, 유망하지 않은 방향까지 비용 순으로 넓게 탐색하므로 느릴 수 있다.

정보 탐색: 휴리스틱, 그리디, A*

정보 탐색은 목표까지 남은 비용을 추정하는 휴리스틱(Heuristic)으로 탐색을 목표 쪽으로 유도한다. 그리디 탐색(Greedy Best - First Search)은 지나온 비용은 무시하고 휴리스틱 추정만 보고 가장 가까워 보이는 노드를 확장해 대개 빠르지만 최적을 보장하지 않는다. A* 탐색(A* Search)은 지나온 실제 비용과 목표까지의 휴리스틱 추정을 더한 값으로 노드를 평가해, 목표를 향하면서도 저렴한 경로를 우선한다. 휴리스틱이 실제 비용을 과대평가하지 않으면 A*는 최적 경로를 보장한다. 균일 비용 탐색은 지나온 비용만, 그리디는 휴리스틱만, A*는 둘 다 쓴다고 정리하면 된다.

적대적 탐색: 미니맥스와 알파-베타 가지치기

체스나 바둑처럼 상대가 있는 게임에서는 적대적 탐색을 쓴다. 미니맥스(Minimax)는 내 차례엔 점수를 최대화하고 상대 차례엔 상대가 내 점수를 최소화한다고 가정해, 게임 트리를 펼쳐 최선의 수를 거슬러 계산한다. 상대가 완벽하게 둔다는 최악의 가정 아래 최선을 고르는 것이다. 그러나 트리가 폭발적으로 커지므로, 알파 - 베타 가지치기(Alpha - Beta Pruning)로 결과에 영향을 주지 않는 가지를 잘라 같은 최적 수를 훨씬 적은 계산으로 찾는다. 관련 용어: 너비 우선 탐색(BFS), 깊이 우선 탐색(DFS), 균일 비용 탐색, 휴리스틱, 그리디 탐색, A* 탐색, 미니맥스, 알파 - 베타 가지치기.

13장. 자연어 처리 심화: 임베딩에서 생성까지

단어 임베딩에서 문맥 임베딩으로

초기 단어 임베딩인 워드투벡(Word2Vec)은 '비슷한 문맥의 단어는 비슷한 의미'라는 아이디어로 각 단어를 벡터로 표현해, '왕 - 남자 + 여자 $\approx$ 여왕' 같은 의미 연산을 가능하게 했다. 그러나 단어마다 고정된 하나의 벡터를 주기 때문에 문맥에 따라 뜻이 달라지는 다의어를 구분하지 못한다. 이 한계는 문맥을 반영해 같은 단어도 상황에 따라 다른 벡터로 표현하는 트랜스포머 기반 임베딩으로 극복되었다. 이것이 오늘날 검색과 RAG에 쓰이는 문장 임베딩의 기반이다.

트랜스포머의 순서 문제와 위치 인코딩

트랜스포머는 셀프 어텐션으로 모든 토큰을 동시에 처리해 병렬화에 강하지만, 그 대가로 순환 신경망과 달리 토큰의 순서를 자체적으로 알지 못한다. 위치 인코딩(Positional Encoding)은 각 토큰의 위치를 나타내는 벡터를 임베딩에 더해, 모델이 '몇 번째 토큰인지'를 알게 한다. 사인·코사인 함수로 정한 고정 방식과 학습으로 얻는 방식이 있다. 덕분에 트랜스포머는 병렬 처리의 이점을 유지하면서도 단어 순서가 중요한 언어를 제대로 다룰 수 있다.

인코더와 디코더, 그리고 두 종류의 어텐션

인코더 - 디코더(Encoder - Decoder) 구조에서 인코더는 입력을 의미 표현으로 압축하고, 디코더는 그 표현을 바탕으로 출력을 한 토큰씩 생성한다. 번역처럼 입력과 출력 길이가 다른 문제에 적합하다. 여기서 두 종류의 어텐션이 구분된다. 셀프 어텐션은 한 시퀀스가 자기 자신을 참조해 문맥을 파악하고, 크로스 어텐션(Cross - Attention)은 디코더가 인코더의 표현을 참조하듯 서로 다른 두 시퀀스를 잇는다. 참고로 GPT 계열 생성 모델은 디코더만, BERT 계열 이해 모델은 인코더만 쓰기도 한다.

생성의 탐색: 그리디와 빔 서치

언어 모델이 문장을 생성할 때 매 단계 가장 확률 높은 토큰 하나만 고르면(그리디 방식) 당장은 좋아 보여도 전체 문장이 어색해질 수 있다. 빔 서치(Beam Search)는 매 단계 확률이 높은 상위 k 개의 부분 문장을 함께 유지하며 확장해, 전체 확률이 더 높은 문장을 찾는다. 빔 크기가 클수록 품질이 오르지만 계산이 느다. 번역이나 요약처럼 출력 품질이 중요한 생성에 널리 쓰인다. 이는 앞서 본 탐색 알고리즘의 아이디어가 언어 생성에 적용된 예이기도 하다. 관련 용어: 워드투벡, 임베딩, 위치 인코딩, 인코더 - 디코더, 셀프 어텐션, 크로스 어텐션, 빔 서치.